SIC AI Chapter Quiz: 05

Student Name: Devesh Panwar

Candidate ID: SIC202500060

Question 01

Option 4: Recall ratio is calculated as $\frac{tp}{tp+fp}$.

Question 02

```
from sklearn.linear_model import LinearRegression
import numpy as np

# Sample Data
X = np.array([[1], [2], [3]])
y = np.array([2, 4, 6])

# Create Model
model = LinearRegression()

# Fitting the model uses the "Least Squares" method to find the best weights
model.fit(X, y)

print(f"Estimated Coefficient (Slope): {model.coef_[0]}")
print(f"Estimated Intercept: {model.intercept_}")
```

```
Estimated Coefficient (Slope): 1.9999999999999996
Estimated Intercept: 8.881784197001252e-16
```

Question 03

Option 4: The cross-validation method is a method to reselect training data repeatedly and based on restoration extraction, and it is suitable when the total amount of data is not large.

```
import numpy as np
from sklearn.model_selection import KFold
from sklearn.utils import resample

# Sample data: 5 items
data = np.array([1, 2, 3, 4, 5])

print("--- 1. Cross-Validation (K-Fold) ---")
print("Notice: No overlap in test sets, no replacement.")
kf = KFold(n_splits=5)
for i, (train_index, test_index) in enumerate(kf.split(data)):
    print(f"Fold {i+1} Test Set: {data[test_index]}")

print("\n--- 2. Bootstrap (Restoration Extraction) ---")
print("Notice: Random selection WITH replacement (duplicates possible).")
# Create 3 bootstrap samples
for i in range(3):
    boot = resample(data, replace=True, n_samples=5, random_state=i)
    print(f"Bootstrap Sample {i+1}: {boot}")
```

```
--- 1. Cross-Validation (K-Fold) ---
Notice: No overlap in test sets, no replacement.
Fold 1 Test Set: [1]
Fold 2 Test Set: [2]
Fold 3 Test Set: [3]
Fold 4 Test Set: [4]
Fold 5 Test Set: [5]

--- 2. Bootstrap (Restoration Extraction) ---
Notice: Random selection WITH replacement (duplicates possible).
Bootstrap Sample 1: [5 1 4 4 4]
Bootstrap Sample 2: [4 5 1 2 4]
Bootstrap Sample 3: [1 1 4 3 4]
```

Question 04

Option 3: CART(Classification and Regression Trees) algorithm is an algorithm that performs separation using the Gini index, and 100 represents a perfectly pure node between 0 and 100.

Question 05

Answer: You should Increase Gamma (γ) and Increase C.

The problem states that the SVM model is underfitting the training set. "Underfitting" means the model is too simple; it is treating the data too broadly and failing to capture the specific patterns or decision boundaries (High Bias).To fix underfitting, we need to increase the model's complexity so it fits the training data more tightly.

```
from sklearn.svm import SVC

# Current Model (Underfitting)
# Low C and Low Gamma make the model too "stiff"
model_underfit = SVC(kernel='rbf', C=0.1, gamma=0.001)

# Improved Model (Fixing Underfitting)
# We INCREASE both values to allow the model to fit the data better
model_better = SVC(kernel='rbf', C=100, gamma=0.1)
```

Question 06

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn import metrics

# 1. Load Data
data = load_breast_cancer()
X = data['data']
# Note: The question code inverts the target (0=Benign, 1=Malignant)
Y = 1 - data['target']

# 2. Split Data (as specified in the image)
# random_state=1234 guarantees we get the same result every time
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.4, random_state=1234)

# 3. Create and Train KNN Model
# "Set the adjacency value to 5" means n_neighbors=5
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, Y_train)

# 4. Make Predictions
Y_pred = knn.predict(X_test)

# 5. Calculate Metrics
accuracy = metrics.accuracy_score(Y_test, Y_pred)
recall = metrics.recall_score(Y_test, Y_pred)
precision = metrics.precision_score(Y_test, Y_pred)

print(f"Accuracy:  {accuracy:.4f}")
print(f"Recall:    {recall:.4f}")
print(f"Precision: {precision:.4f}")
```

```
Accuracy:  0.9254
Recall:    0.8571
Precision: 0.9351
```